

Universidad Técnica Estatal

Facultad de Ciencias de la Computación y Diseño Digital

Ingeniería en Software

Práctica Experimental – Unidad 1

Comparación de Mecanismos de Comunicación entre Procesos en un Sistema Distribuido: Sockets TCP y gRPC con Protocol Buffers

Integrantes:

Cristhian Daniel Pacheco Cárdenas

Robinson Rodrigo Cando Moreno

Ernesto Gregory Luna Mora

Carrera:

Ingeniería en Software

Profesor:

Gleiston Ciceron Guerrero Ulloa

27 de mayo de 2026

Resumen

Esta práctica experimental implementa y compara dos mecanismos de comunicación entre procesos en un sistema distribuido de tres nodos: Sockets TCP y gRPC con Protocol Buffers. Cada nodo se ejecuta como un proceso independiente de Python en la misma máquina, escuchando en los puertos 5000, 5001 y 5002 para TCP, y 6000, 6001 y 6002 para gRPC. Se implementó el reloj lógico de Lamport para ordenar los eventos del sistema, siguiendo la regla $\text{máx}(\text{local}, \text{recibido}) + 1$. Se ejecutaron 20 intercambios entre nodos y se midió la latencia promedio de 100 envíos usando `time.perf_counter()`. Los resultados obtenidos muestran que TCP presentó una latencia promedio de 2.6357 ms, mientras que gRPC registró 5.4692 ms, siendo TCP el protocolo más rápido en este entorno local.

Palabras clave: sistemas distribuidos, sockets TCP, gRPC, Protocol Buffers, reloj de Lamport, latencia.

Abstract

This experimental practice implements and compares two interprocess communication mechanisms in a distributed system of three nodes: TCP Sockets and gRPC with Protocol Buffers. Each node runs as an independent Python process on the same machine, listening on ports 5000, 5001, and 5002 for TCP, and 6000, 6001, and 6002 for gRPC. Lamport's logical clock was implemented to order system events, following the rule $\text{máx}(\text{local}, \text{received}) + 1$. Twenty exchanges were executed between nodes, and the average latency of 100 sends was measured using `time.perf_counter()`. Results show that TCP achieved an average latency of 2.6357 ms, while gRPC recorded 5.4692 ms, making TCP the faster protocol in this local environment.

Keywords: distributed systems, TCP sockets, gRPC, Protocol Buffers, Lamport clock, latency.

Índice

1. Introducción	3
2. Marco Teórico	3
2.1. Sistemas distribuidos	3
2.2. Comunicación entre procesos	3
2.3. Sockets TCP	4
2.4. gRPC y Protocol Buffers	4
2.5. Reloj lógico de Lamport	4
3. Configuración del Entorno de Desarrollo	4
4. Implementación con Sockets TCP	5
5. Implementación con gRPC y Protocol Buffers	6
5.1. Definición del contrato en nodo.proto	6
5.2. Servidor y cliente gRPC	7
6. Módulo Comparador	7
7. Proceso de Ejecución	7
8. Resultados	9
8.1. Comportamiento del reloj de Lamport	9
8.2. Latencias obtenidas con TCP	9
8.3. Latencias obtenidas con gRPC	9
9. Análisis de Resultados	10
9.1. Comparación de latencias	10
9.2. Funcionamiento del reloj de Lamport	11
9.3. Facilidad de implementación	11
10.Repositorio en GitHub	12
11.Conclusiones Generales	13

1. Introducción

Los sistemas distribuidos son aquellos en los que varios procesos trabajan juntos, comunicándose entre sí para llevar a cabo una tarea común. En este tipo de sistemas, la forma en que los procesos se comunican es fundamental, ya que afecta directamente al rendimiento, la escalabilidad y la complejidad del sistema.

En esta práctica experimental se implementó y comparó dos mecanismos distintos de comunicación entre procesos: los **Sockets TCP** y **gRPC con Protocol Buffers**. El objetivo fue entender cómo funcionan ambas tecnologías, implementarlas en un entorno de tres nodos que se ejecutan simultáneamente en una misma máquina, y medir su rendimiento en términos de latencia.

Para controlar el orden de los mensajes en el sistema distribuido, se utilizó el **reloj lógico de Lamport** [1], que permite ordenar los eventos sin depender de un reloj físico global. El desarrollo se realizó en **Python**, usando **IntelliJ IDEA Ultimate**, y el código se gestionó en un repositorio de GitHub con al menos tres contribuciones por integrante.

2. Marco Teórico

2.1. Sistemas distribuidos

Un sistema distribuido es un conjunto de nodos autónomos que se comunican mediante el intercambio de mensajes y que, desde el punto de vista del usuario, se comportan como un sistema único [5]. En esta práctica, los tres nodos son procesos de Python que escuchan en puertos distintos de la misma máquina, simulando nodos independientes en una red local.

2.2. Comunicación entre procesos

La comunicación entre procesos (IPC) es el mecanismo por el cual los procesos de un sistema distribuido intercambian información [2]. Elegir el mecanismo adecuado es una decisión importante, ya que afecta el tiempo de respuesta, el uso de recursos y la complejidad del código [3]. En esta práctica se comparan dos enfoques: los Sockets TCP, que operan a bajo nivel, y gRPC, que ofrece una abstracción de alto nivel.

2.3. Sockets TCP

Los Sockets TCP son una forma de comunicación basada en el protocolo de control de transmisión, que garantiza la entrega completa y ordenada de los datos. Al usarlos directamente, el programador controla por completo cómo se construyen, envían y reciben los mensajes. En esta práctica, los mensajes se codifican en formato JSON con la estructura `{sender, timestamp, message}`.

2.4. gRPC y Protocol Buffers

gRPC es un framework desarrollado por Google que facilita la comunicación entre servicios distribuidos. El contrato del servicio se define en un archivo `.proto`, a partir del cual se genera automáticamente el código de cliente y servidor. Los Protocol Buffers serializan los datos en formato binario, más compactos que el JSON. Estudios recientes han demostrado que gRPC tiende a tener menores tiempos de respuesta en arquitecturas de microservicios [4].

2.5. Reloj lógico de Lamport

El reloj de Lamport fue propuesto en 1978 para asignar un orden a los eventos en un sistema distribuido sin necesidad de un reloj físico común [1]. Cada nodo mantiene un contador entero que se actualiza con dos reglas:

- **Al enviar:** el contador local se incrementa en 1.
- **Al recibir:** se aplica la fórmula:

$$\text{reloj} = \text{máx}(\text{reloj_local}, \text{reloj_recibido}) + 1$$

Gracias a esta regla, es posible establecer un orden coherente de los eventos a pesar de que los nodos no compartan un reloj físico [6].

3. Configuración del Entorno de Desarrollo

Para el desarrollo de esta práctica se utilizaron las siguientes herramientas: **Python 3.12** como lenguaje de programación, **IntelliJ IDEA Ultimate** con los plugins de Python y Protocol Buffers instalados, las librerías **grpcio** y **grpcio-tools** para implementar gRPC, y **Git con GitHub** para el control de versiones y la colaboración.

El proyecto quedó organizado como se muestra en la Figura 1, donde se pueden observar los archivos principales del sistema: `nodo_tcp.py`, `nodo_grpc.py`, `nodo.proto`, `comparador.py` y los archivos CSV de resultados. Los archivos `nodo_pb2.py` y `nodo_pb2_grpc.py` fueron generados automáticamente al ejecutar el compilador de Protocol Buffers sobre el archivo `proto`.

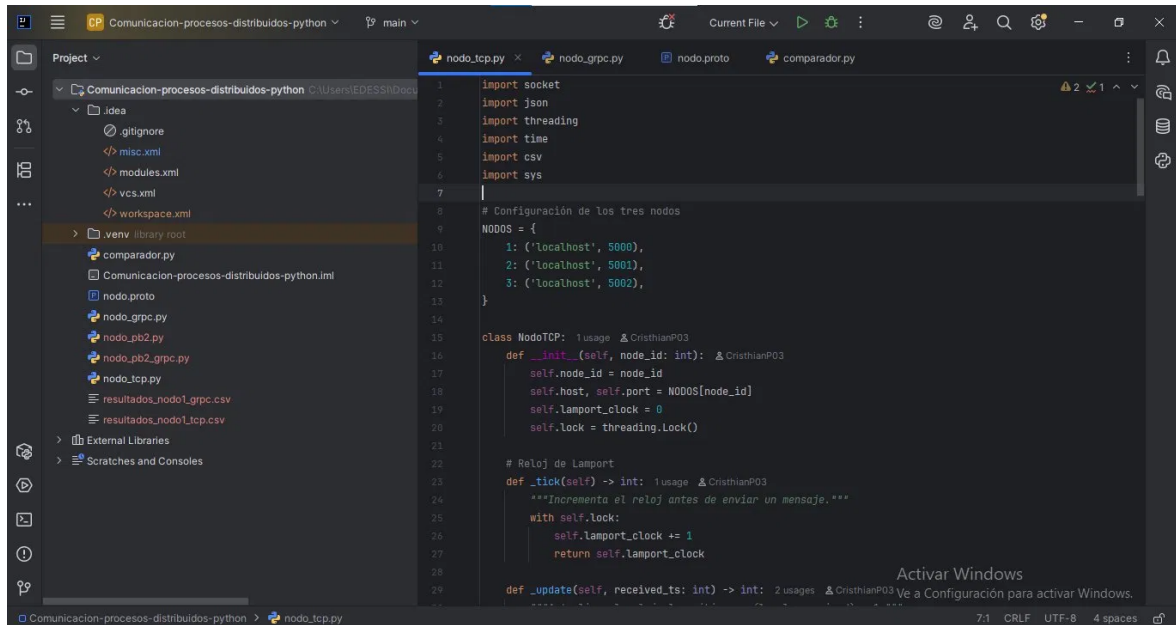


Figura 1: Estructura del proyecto en IntelliJ IDEA

4. Implementación con Sockets TCP

La implementación TCP se realizó en `nodo_tcp.py`. Al ejecutarlo con el argumento 1, 2 o 3, el proceso abre un servidor en el puerto correspondiente (5000, 5001 o 5002) y puede también actuar como cliente enviando mensajes a los otros nodos.

Cada nodo mantiene un reloj de Lamport protegido por un `threading.Lock()` para evitar errores de concurrencia. Los dos métodos centrales del reloj son los siguientes

```
def _tick(self):
    with self.lock:
        self.lamport_clock += 1
    return self.lamport_clock

def _update(self, received_ts):
    with self.lock:
        self.lamport_clock = max(
            self.lamport_clock, received_ts
        ) + 1
    return self.lamport_clock
```

El servidor corre en un hilo en segundo plano. Al recibir un mensaje JSON, extrae el timestamp del emisor, actualiza el reloj con `_update()` y responde con su propio timestamp actualizado. El cliente llama a `_tick()` antes de enviar, construye el JSON y abre la conexión TCP al nodo destino. La medición de latencia se realizó sobre 100 envíos usando `time.perf_counter()`, y los resultados se guardan en `resultados_nodo1_tcp.csv`.

5. Implementación con gRPC y Protocol Buffers

5.1. Definición del contrato en `nodo.proto`

El primer paso fue definir el servicio en el archivo `nodo.proto`, que especifica los mensajes intercambiados y el método disponible:

```
syntax = "proto3";

service NodoService {
    rpc Enviar (Mensaje) returns (Respuesta);
}

message Mensaje {
    string sender      = 1;
    int64  timestamp  = 2;
    string message     = 3;
}

message Respuesta {
    string responder = 1;
    int64  timestamp = 2;
}
```

5.2. Servidor y cliente gRPC

La implementación en `nodo_grpc.py` sigue la misma lógica de Lamport que TCP. El servidor hereda de `NodoServiceServicer` y sobrescribe el método `Enviar()`, que recibe el mensaje, actualiza el reloj y devuelve la respuesta. El cliente abre un canal hacia el nodo destino, obtiene un *stub* y llama al método remoto como si fuera local, sin manejar directamente sockets ni serialización.

La principal diferencia con TCP es que gRPC delega toda la comunicación al framework, simplificando el código. Los puertos usados para gRPC fueron 6000, 6001 y 6002, distintos a los de TCP para evitar conflictos.

6. Módulo Comparador

Para analizar y comparar los resultados de ambas implementaciones, se desarrolló `comparador.py`. Este módulo lee los dos archivos CSV generados y calcula el promedio, mínimo y máximo de latencia de cada protocolo, mostrando al final cuál fue el más rápido. Su ejecución es independiente de los nodos y solo requiere que los archivos CSV existan previamente.

7. Proceso de Ejecución

Para ejecutar cualquiera de las dos implementaciones, se abren tres terminales simultáneas y se inician primero los nodos 2 y 3, ya que el nodo 1 espera 3 segundos automáticamente antes de comenzar. Una vez listos, el Nodo 1 ejecuta los 20 intercambios con reloj de Lamport, luego los 100 envíos de latencia y finalmente genera el archivo CSV.

La Figura 2 muestra la salida de la terminal durante la ejecución TCP, donde se puede observar el avance del reloj de Lamport en cada intercambio, alcanzando un valor de 60 al finalizar los 20 intercambios, y la latencia promedio obtenida de 2.6357 ms.


```

1 import csv
2 import sys
3
4 # Configuración de los tres nodos
5 NODOS = {
6     1: ('localhost', 5000),
7     2: ('localhost', 5001),
8     3: ('localhost', 5002),
9 }
10
11 class NodeTCP:
12     def __init__(self, node_id: int):
13         self.node_id = node_id
14         self.host, self.port = NODOS[node_id]
15         self.lamport_clock = 0
16         self.lock = threading.Lock()
17
18 # Reloj de Lamport
19 def tick(self) -> int:
20     """Incrementa el reloj antes de enviar"""
21     with self.lock:
22         self.lamport_clock += 1
23     return self.lamport_clock
24
25 def update(self, received_ts: int) -> int:
26     """Actualiza el reloj al recibir mensaje"""
27     with self.lock:
28         self.lamport_clock = max(self.lamport_clock, received_ts + 1)
29     return self.lamport_clock
30
31 # Servidor (método del Servidor)
32 def main():
33     # Crear los tres nodos
34     n1 = NodeTCP(1)
35     n2 = NodeTCP(2)
36     n3 = NodeTCP(3)
37
38     # Ejecutar los 20 intercambios
39     for i in range(20):
40         # Enviar mensaje de n1 a n2
41         ts1 = n1.tick()
42         n2.update(ts1)
43
44         # Enviar mensaje de n2 a n3
45         ts2 = n2.tick()
46         n3.update(ts2)
47
48         # Enviar mensaje de n3 a n1
49         ts3 = n3.tick()
50         n1.update(ts3)
51
52     # Guardar resultados
53     with open('resultados_nodo1_tcp.csv', 'w') as f:
54         f.write('Nodo, Reloj, Latencia\n')
55         for i in range(20):
56             f.write(f'{i+1}, {n1.lamport_clock}, {n1.lamport_clock - n1.lamport_clock_initial}\n')
57
58     print(f'Latencia promedio: 2.6357 ms')
59
60 if __name__ == '__main__':
61     main()

```

Figura 2: Ejecución de los 20 intercambios TCP con reloj de Lamport

De manera similar, la Figura 3 muestra la ejecución del protocolo gRPC, con el mismo patrón de intercambios y una latencia promedio de 5.4692 ms.

```

1 import grpc
2 import time
3 import sys
4 import threading
5 from concurrent import futures
6
7 import nodedb_pb2
8 import nodedb_pb2_grpc
9
10 PUERTOS = {1: 6000, 2: 6001, 3: 6002}
11
12 class NodeGRPC(nodedb_pb2_grpc.NodeServiceServicer):
13     def __init__(self, node_id: int):
14         self.node_id = node_id
15         self.lamport = 0
16         self.lock = threading.Lock()
17
18 # Reloj de Lamport
19 def tick(self):
20     """Incrementa el reloj antes de enviar"""
21     with self.lock:
22         self.lamport += 1
23     return self.lamport
24
25 def update(self, received_ts: int):
26     """Actualiza el reloj al recibir mensaje"""
27     with self.lock:
28         self.lamport = max(self.lamport, received_ts + 1)
29     return self.lamport
30
31 # Servidor (método del Servicer)
32 def main():
33     # Crear los tres nodos
34     n1 = NodeGRPC(1)
35     n2 = NodeGRPC(2)
36     n3 = NodeGRPC(3)
37
38     # Ejecutar los 20 intercambios
39     for i in range(20):
40         # Enviar mensaje de n1 a n2
41         ts1 = n1.tick()
42         n2.update(ts1)
43
44         # Enviar mensaje de n2 a n3
45         ts2 = n2.tick()
46         n3.update(ts2)
47
48         # Enviar mensaje de n3 a n1
49         ts3 = n3.tick()
50         n1.update(ts3)
51
52     # Guardar resultados
53     with open('resultados_nodo1_grpc.csv', 'w') as f:
54         f.write('Nodo, Reloj, Latencia\n')
55         for i in range(20):
56             f.write(f'{i+1}, {n1.lamport}, {n1.lamport - n1.lamport_initial}\n')
57
58     print(f'Latencia promedio: 5.4692 ms')
59
60 if __name__ == '__main__':
61     main()

```

Figura 3: Ejecución de los 20 intercambios gRPC con reloj de Lamport

8. Resultados

8.1. Comportamiento del reloj de Lamport

En ambas implementaciones, el reloj de Lamport avanzó correctamente durante los 20 intercambios. Cada mensaje enviado incrementaba el reloj del emisor, y al recibirlo, el receptor aplicaba la regla $\max(\text{local}, \text{recibido}) + 1$, asegurando un orden coherente de los eventos. En ambos protocolos el reloj alcanzó el valor 60 al finalizar los 20 intercambios, lo que confirma el correcto funcionamiento del algoritmo.

8.2. Latencias obtenidas con TCP

Los resultados de la medición de latencia TCP se almacenaron en el archivo `resultados_nodo1_tcp.csv`, cuyo contenido se puede observar en la Figura 4. Los valores obtenidos fueron los siguientes:

- **Promedio:** 2.6357 ms
- **Mínimo:** 1.5026 ms
- **Máximo:** 13.2911 ms

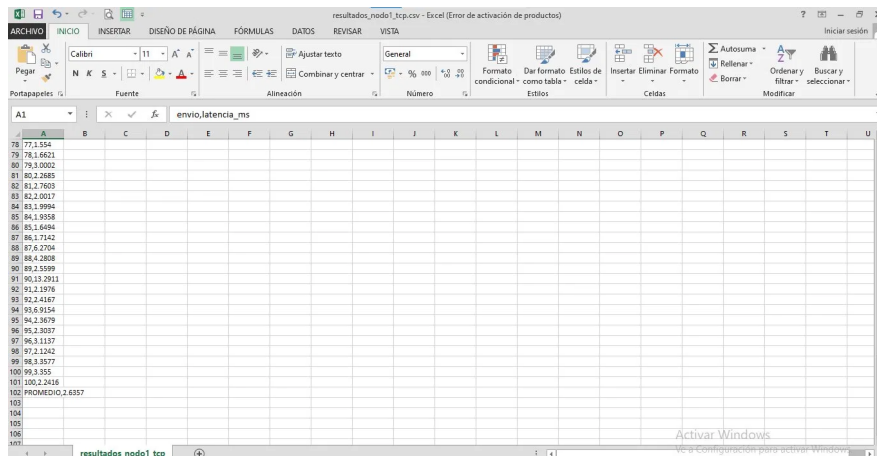


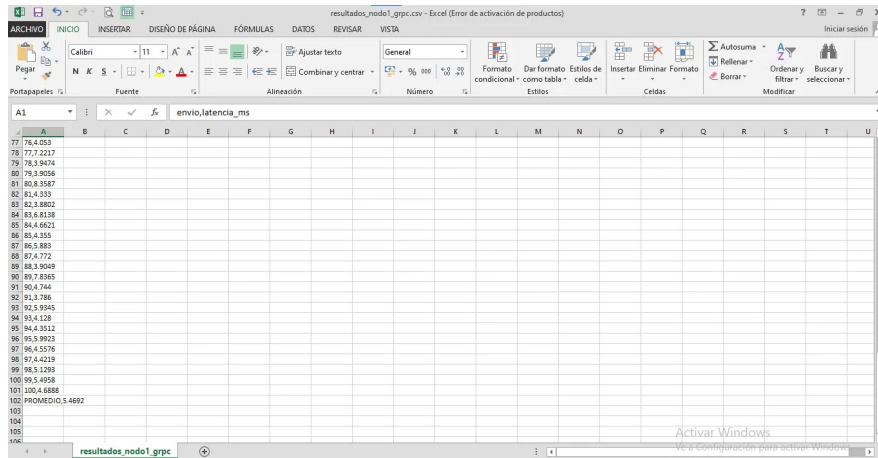
Figura 4: Archivo CSV con las latencias de los 100 envíos TCP

8.3. Latencias obtenidas con gRPC

Los resultados de la medición de latencia gRPC se almacenaron en el archivo `resultados_nodo1_grpc.csv`, cuyo contenido se puede observar en la Figura 5. Los valores obtenidos fueron los siguientes:

- **Promedio:** 5.4692 ms

- **Mínimo:** 3.3481 ms
- **Máximo:** 18.8438 ms



envio,latencia_ms
77.764093
78.7772217
79.7839474
80.7939056
81.8083587
82.814335
83.8238802
84.8368188
85.846621
86.854355
87.865883
88.874772
89.8839049
90.8978365
91.904744
92.913786
93.9250345
94.934128
95.9443512
96.9550923
97.9645576
98.9744219
99.9851295
100.9954958
101.10046888
102.PROMEDIO:5.4692
103.
104.
105.
106.

Figura 5: Archivo CSV con las latencias de los 100 envíos gRPC

9. Análisis de Resultados

9.1. Comparación de latencias

La Figura 6 muestra la salida del módulo comparador, que resume los resultados de ambos protocolos. TCP obtuvo una latencia promedio de 2.6357 ms frente a los 5.4692 ms de gRPC, siendo TCP aproximadamente dos veces más rápido en este entorno de prueba.

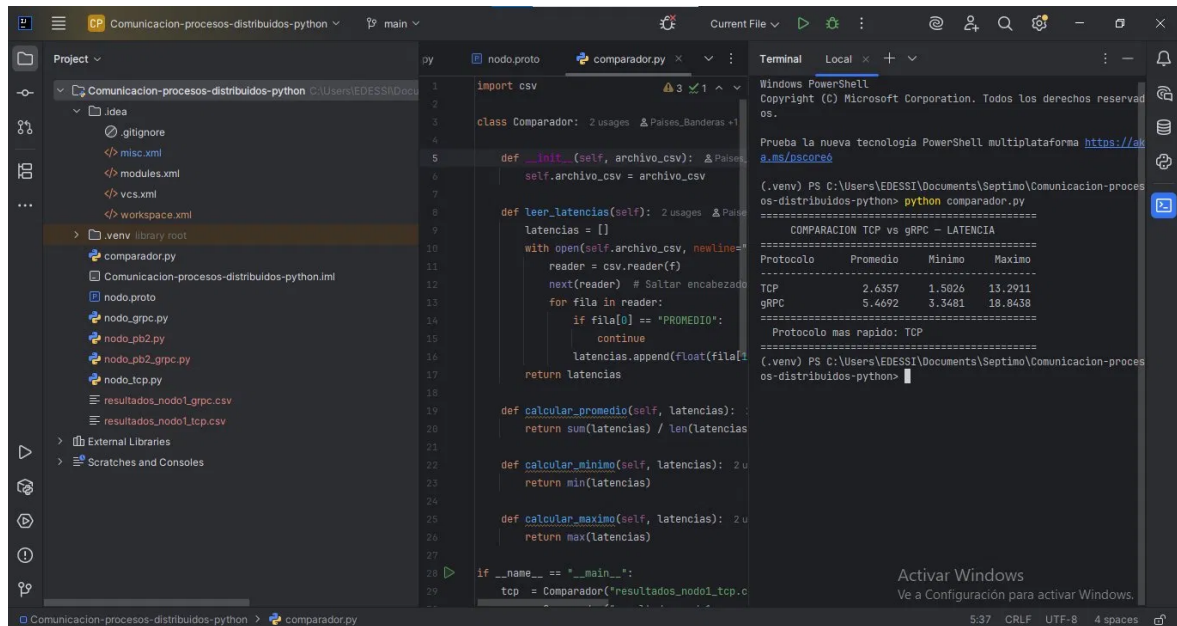


Figura 6: Resultado del comparador mostrando TCP vs gRPC

Esta diferencia puede explicarse porque en un entorno local, la sobrecarga de gRPC al establecer canales HTTP/2 y serializar los datos con Protocol Buffers resulta mayor que la simple apertura de un socket TCP con un mensaje JSON pequeño. En sistemas distribuidos reales con mayor volumen de datos o mayor latencia de red, los Protocol Buffers podrían ofrecer una ventaja al ser más compactos que el JSON, lo que podría invertir estos resultados [4].

9.2. Funcionamiento del reloj de Lamport

El reloj de Lamport funcionó correctamente en ambas implementaciones. Se verificó que en cada intercambio el valor del reloj aumentaba de forma coherente aplicando la regla $\max(\text{local}, \text{recibido}) + 1$. En los 20 intercambios, el reloj avanzó de 0 a 60 en ambos protocolos, lo cual es consistente con lo propuesto originalmente por Lamport [1]: cada envío y cada recepción generan un incremento, lo que garantiza el orden parcial de los eventos.

9.3. Facilidad de implementación

Desde el punto de vista del desarrollo, los Sockets TCP requieren manejar manualmente la serialización de mensajes, la apertura y cierre de conexiones, y los posibles errores de red. gRPC abstrae todos estos detalles al generar el código de comunicación automáticamente desde el archivo `.proto`. Como señalan Fu y Cai [2], la complejidad

de la comunicación entre procesos impacta directamente en la calidad y mantenibilidad del software distribuido. Como señala Birman [6], la latencia es uno de los factores más críticos en el diseño de sistemas distribuidos confiables.

10. Repositorio en GitHub

El código fuente completo de esta práctica se encuentra disponible en: <https://github.com/CristhianP03/Actividades-Grupales-App-Distribuidas.git>

Las contribuciones se distribuyeron de la siguiente manera:

- **Cristhian Pacheco:** configuración del proyecto y `nodo.proto`, servidor TCP con parser JSON, integración del reloj de Lamport en `nodo_tcp.py`.
- **Robinson Cando:** servidor gRPC con `NodoServiceImpl`, cliente gRPC con `send_to()`, métodos de intercambio y latencia en `nodo_grpc.py`.
- **Ernesto Luna:** módulo comparador con lectura de CSV, métodos de cálculo estadístico, análisis comparativo final.

11. Conclusiones Generales

A través de esta práctica se implementaron y compararon dos mecanismos de comunicación en un sistema distribuido de tres nodos. Los resultados muestran que en un entorno local, TCP obtuvo una latencia promedio de 2.6357 ms, menor que los 5.4692 ms registrados por gRPC, siendo TCP el protocolo más rápido en esta prueba.

El reloj lógico de Lamport funcionó correctamente en ambas implementaciones, avanzando de forma coherente de 0 a 60 durante los 20 intercambios, lo que confirma el correcto ordenamiento de los eventos en el sistema distribuido.

Desde el punto de vista del desarrollo, gRPC ofrece mayor facilidad de implementación y mantenimiento al generar el código de comunicación automáticamente, aunque su configuración inicial es más compleja. Los Sockets TCP brindan mayor control y menor sobrecarga en entornos locales con mensajes pequeños.

Conclusiones Individuales

Cristhian Pacheco

Antes de realizar esta práctica, al momento de tener que configurar el proyecto, tener que definir el archivo `nodo.proto` y realizar la implementación del archivo del servidor TCP, me di cuenta de que detrás de cada mensaje que viaja entre los procesos hay muchas decisiones que se van a tomar: desde cómo formatear los datos, cómo abrir las conexiones hasta como tener que cerrar estas mismas, y cómo asegurarse de que los eventos ocurran en el orden correcto. Una de las cosas que más me llamó la atención fue el reloj de Lamport. Al principio me parecía una idea algo complicada, pero cuando lo implementé y vi cómo el número subía en cada intercambio que se iba realizando, entendí que es una solución a un problema real. Los resultados también me hicieron reflexionar, porque pensaba que gRPC podría llegar a ser más rápido que TCP, y no fue así al final.

Robinson Cando

Al momento de tener que realizar la implementación del archivo gRPC, al principio no entendía bien por qué razón había que definir un archivo `.proto` antes de escribir cualquier código, pero luego me di cuenta que ese archivo es como un contrato: le dice a todos los participantes del sistema exactamente qué mensajes van a enviarse y qué van a responder. Eso hace que el código sea más ordenado y menos propenso a errores. Lo que más me sorprendió fue que, a pesar de toda esa estructura, gRPC terminó siendo más lento que TCP en nuestra prueba. Después de investigar un poco, me di cuenta que esa sobrecarga existe porque gRPC hace muchas cosas internamente para garantizar compatibilidad y eficiencia en redes reales. En un escenario con más datos o más nodos, probablemente los resultados serían distintos. Esta práctica me recordó que ninguna tecnología es mejor en todos los contextos.

Ernesto Luna

La parte que me asignaron a mi fue la de construir el comparador y analizar los datos que se generaron en los otros dos módulos. Parece una tarea sencilla, pero me obligó a entender bien cómo funcionaban tanto TCP como gRPC, porque si yo no entendía cómo estaban estructurados los CSV, no podía procesarlos correctamente. Algo que me pareció interesante fue el ver que la latencia no es un número fijo: en el CSV hay envíos que tardaron 1.5 ms y otros que tardaron más de 13 ms, dependiendo de lo que estuviera haciendo el sistema en ese momento. Por eso medir el promedio de los 100 envíos es más difícil que medir uno solo. También aprendí mucho del trabajo en equipo usando GitHub. Coordinar commits con mis compañeros, revisar lo que cada uno aportó y mantener el código organizado es algo que suena fácil pero requiere disciplina.

Referencias

- [1] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978, doi: 10.1145/359545.359563.
- [2] X. Fu and H. Cai, “Measuring interprocess communications in distributed systems,” in *Proc. IEEE/ACM 27th Int. Conf. Program Comprehension (ICPC)*, Montreal, QC, Canada, 2019, pp. 323–334, doi: 10.1109/ICPC.2019.00051.
- [3] C. L. Chamas, D. Cordeiro, and M. M. Eler, “Comparing REST, SOAP, Socket and gRPC in computation offloading of mobile applications: An energy cost analysis,” in *Proc. IEEE 9th Latin-American Conf. Commun. (LATINCOM)*, Guatemala City, Guatemala, Nov. 2017, pp. 1–6, doi: 10.1109/LATINCOM.2017.8240185.
- [4] M. Niswar, R. A. Safruddin, A. Bustamin, and I. Aswad, “Performance evaluation of microservices communication with REST, GraphQL, and gRPC,” *Int. J. Electron. Telecommun.*, vol. 70, no. 2, pp. 429–436, Jun. 2024, doi: 10.24425/ijet.2024.149562.
- [5] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed. CreateSpace Independent Publishing Platform, 2017, ISBN: 978-1-543057-38-6.
- [6] K. Birman, *Guide to Reliable Distributed Systems: Building High-Assurance Applications and Cloud-Hosted Services*. New York, NY, USA: Springer, 2012, doi: 10.1007/978-1-4471-2416-0.